

Contents

1. Estructuras de control clásicas: FSM, Árboles de Comportamiento y Utility AI	1
1.1 FSM y Behavior Trees como estructuras de control eficientes	1
1.2 Limitaciones: explosión combinatoria y la “scripting curse”	1
1.3 Utility AI como respuesta: funciones de utilidad ponderadas	2
1.4 La brecha persistente: reglas y parámetros definidos por el diseñador	2
Referencias nuevas	2

1. Estructuras de control clásicas: FSM, Árboles de Comportamiento y Utility AI

1.1 FSM y Behavior Trees como estructuras de control eficientes

Las Máquinas de Estados Finitos (Finite State Machines, FSM) modelan el comportamiento de un NPC como un grafo de estados discretos —cada uno representa una conducta o configuración interna (patrullar, perseguir, huir)— conectados por transiciones dirigidas que se disparan cuando se cumplen ciertas condiciones o eventos; en todo momento un único estado está “activo” [9]. Su atractivo técnico radica en que son deterministas, fáciles de esbozar y visualizar, y de bajo costo computacional, lo que las convierte en una elección práctica para el control de NPCs en producción [9]. Los Árboles de Comportamiento (Behavior Trees, BT), popularizados por Damian Isla en Halo 2 (2005), estructuran la conducta de forma jerárquica: un nodo raíz envía señales periódicas (“ticks”) que descienden por el árbol, y cada nodo devuelve uno de tres estados —running, success o failure— [9]. Los nodos de flujo de control (secuencia, fallback/selector, paralelo y decorador) deciden qué hijos ejecutar y en qué orden, mientras los nodos de ejecución (acción y condición) hacen el trabajo en las hojas [9]. A diferencia de la FSM, la lógica de transición no se dispersa dentro de cada estado, sino que se organiza en la estructura del árbol, lo que mejora la modularidad y la reactividad: como el árbol se re-evalúa desde la raíz en cada tick, una rama de mayor prioridad (p. ej. “huir con salud baja”) interrumpe naturalmente a las inferiores sin necesidad de cablear una transición desde cada estado [9], [12]. Ambos enfoques comparten una virtud central: son baratos de ejecutar y predecibles, requisitos indispensables para correr decenas o cientos de agentes a 30 Hz o más [8].

1.2 Limitaciones: explosión combinatoria y la “scripting curse”

El costo de esa eficiencia aparece al escalar la complejidad. Isla documentó que, en Halo 2, ajustar floats de relevancia para obtener un conjunto específico de reglas o prioridades “es factible cuando hay dos o tres opciones, pero cuando hay veinte o más es casi imposible” [8]. Describió además el fenómeno de “parameter creep”: “tomen tres parámetros, por unos 115 comportamientos, por unos 30 tipos de personaje (incluidos los tipos fundamentales y sus variantes, como élitos rojos frente a blancos) y tenemos cerca de 10,350 números distintos que mantener” [8]. Combinados, estos factores producen “una explosión de floats inescrutables. ¿Cuál de los cientos de números potenciales es el que hace que un enemigo en particular ‘se sienta mal’?” [8]. En las FSM, agregar o quitar estados obliga a re-evaluar un número potencialmente grande de transiciones, lo que las hace frágiles ante el error humano y difíciles de mantener [9]. Colledanchise y Ögren señalan que “los métodos de diseño tradicionales, como las máquinas de estados finitos, son conocidos por producir comportamientos frágiles cuando la complejidad aumenta, haciendo muy difícil agregar características sin romper la funcionalidad existente” [9]; cada transición crea una dependencia entre dos módulos, de modo que con N estados el número de transiciones posibles crece de forma cuadrática [9]. Este problema se conoce en la literatura como explosión combinatoria o state-space explosion de estados y transiciones [12]. El término coloquial “scripting curse” o “maldición del scripting” no aparece como terminología establecida en las fuentes académicas revisadas y debe tratarse como una expresión divulgativa [sin confirmar]; el concepto verificable subyacente es precisamente la explosión combinatoria y el hecho de que estas estructuras son scripts escritos a mano cuya eficacia depende de la previsión del diseñador: “sobresalen en escenarios predecibles pero no pueden aprender ni adaptarse más allá de lo explícitamente codificado” y “se romperían o requerirían reautoría” ante un cambio imprevisto del entorno [13]. Es exactamente esta limitación la que la fuente primaria de esta tesis identifica: a diferencia de los modelos de FSM tradicionales, que suelen fallar cuando falta una variable ambiental esperada, su arquitectura GOAP modular separa las metas de alto nivel de las acciones procedimentales para permitir cambios e interrupciones de tarea [1].

1.3 Utility AI como respuesta: funciones de utilidad ponderadas

La IA de utilidad (Utility AI) surge para flexibilizar esta rigidez. En lugar de transiciones binarias predefinidas, cada acción posible se puntúa numéricamente y se elige una de las mejores [11]. La idea central es que toda acción o estado puede describirse con un valor uniforme de utilidad que mide su conveniencia en el contexto actual; las puntuaciones se normalizan al rango [0,1] para poder combinarse [11]. Cada decisión se descompone en “decision factors” (factores de consideración) que se transforman mediante curvas de respuesta —lineal, cuadrática, logística o lineal por tramos— y se ponderan según cuánto “le importa” ese factor al agente, emulando personalidad [11]. La elección final aplica el principio de máxima utilidad esperada [11]. Dave Mark formalizó este patrón en su libro *Behavioral Mathematics for Game AI* [10] y, junto con Mike Lewis (ArenaNet), presentó una arquitectura autónoma completa —el Infinite Axis Utility System (IAUS)— en el AI Summit de la GDC 2015 [14]. En esa arquitectura los sistemas de comportamiento “son menos rígidos y, por lo tanto, menos frágiles. La IA rara vez —o nunca— queda ‘atascada’ en un comportamiento del que no debería porque no puede salir de lo que está haciendo” [14]. El resultado es un comportamiento continuo y sensible al contexto, computacionalmente barato (sumas y multiplicaciones) y escalable. La fuente primaria incorpora esta lógica al ponderar dinámicamente metas: cada meta recibe un peso base modificado en tiempo real por los estados del mundo y del NPC, y el peso total determina la prioridad del agente [1].

1.4 La brecha persistente: reglas y parámetros definidos por el diseñador

Aun así, el razonamiento del agente de Utility AI sigue dependiendo de reglas y parámetros definidos manualmente por el diseñador. Como advierte la literatura técnica, calcular la utilidad inicial de un factor “es altamente subjetivo: dos programadores escribirán dos funciones de utilidad distintas que producen salidas distintas ante las mismas entradas”, y dar con la función adecuada “es más arte que ciencia” [11]. Las curvas, los pesos y las consideraciones son datos que alguien debe autorar y afinar a mano; funciones mal definidas producen oscilaciones y decisiones contextualmente inapropiadas. En otras palabras, Utility AI amplía enormemente la flexibilidad del comportamiento, pero no elimina la dependencia del conocimiento explícito del diseñador ni dota al agente de deliberación semántica sobre situaciones no contempladas en su diseño. Esta brecha —entre la ejecución táctica eficiente y una deliberación flexible que no requiera anticipar cada caso— es la que motiva el resto de esta tesis y su propuesta de integrar deliberación semántica mediante modelos de lenguaje compactos con ejecución táctica (GOAP).

Referencias nuevas

Fuente primaria obligatoria (ya presente en la bibliografía del anteproyecto como [1]):

[1] Jacinto, J. A. B., Sanig, M. K. S., Namuag, S. G., & Olmiguez II, E. D. (2024). Development of Autonomous NPC Worker Agents through a Modular GOAP Architecture in 3D Simulations. *International Journal on Science and Technology (IJSAT)*. [Texto completo consultado en ijsat.org, Vol. 17, Issue 1, 2026.]

Fuentes complementarias añadidas por esta investigación (numeración propuesta a partir de [8]):

[8] Isla, D. (2005). Handling Complexity in the Halo 2 AI. *Proceedings of the 2005 Game Developers Conference (GDC)*.

[9] Colledanchise, M., & Ögren, P. (2018). *Behavior Trees in Robotics and AI: An Introduction*. CRC Press. (Preprint: [arXiv:1709.00084](https://arxiv.org/abs/1709.00084)).

[10] Mark, D. (2009). *Behavioral Mathematics for Game AI*. Charles River Media.

[11] Graham, D. “Rez” (2014). *An Introduction to Utility Theory*. En S. Rabin (Ed.), *Game AI Pro*. CRC Press.

[12] Iovino, M., Scukins, E., Styruud, J., Ögren, P., & Smith, C. (2022). A survey of Behavior Trees in robotics and AI. *Robotics and Autonomous Systems*, 154, 104096.

[13] [Autores sin confirmar] (2025). NPC Control Across Algorithm Classes: Static Logic, Evolutionary, Reinforcement Learning, and Hybrid Generative Methods. *Proceedings of the 2025 8th International Conference on Computational Intelligence and Intelligent Systems (CIIS '25)*, ACM. DOI: 10.1145/3787256.3787265.

[14] Mark, D. (Intrinsic Algorithm). *The Infinite Axis Utility System (IAUS)*. gameai.com. [Presentado en el AI Summit, GDC 2013 y 2015.]

Nota de verificación de fuentes: se consultaron en texto completo la fuente primaria [1] (PDF en ijsat.org), el proceeding de Isla [8] (gamedeveloper.com), el capítulo de Utility Theory [11] (gameaipro.com) y el preprint del libro de Colledanchise y Ögren [9] (arXiv:1709.00084). El libro de Mark [10] se verificó por índice y reseñas, no en texto completo; del paper ACM [13] se confirmaron citas verbatim pero **no los nombres de autores [sin confirmar]** (verificar en dl.acm.org/doi/10.1145/3787256.3787265). El término “scripting curse”/“maldición del scripting” **no pudo verificarse como terminología académica establecida** y se marcó como [sin confirmar]; el concepto documentado equivalente es la explosión combinatoria/state-space explosion de estados y transiciones.